8.5.4 In-Place Selection

In-place, nondestructive, selection is conceptually simple, but it requires a lot of bookkeeping, and it is correspondingly slow. The general idea is to pick some number M of elements at random, to sort them, and then to make a pass through the array *counting* how many elements fall in each of the $M + 1$ intervals defined by these elements. The kth largest will fall in one such interval — call it the "live" interval. One then does a second round, first picking M random elements in the live interval, and then determining which of the new, finer, $M + 1$ intervals all presently live elements fall into. And so on, until the kth element is finally localized within a single array of size M, at which point direct selection is possible.

How shall we pick M? The number of rounds, $\log_M N = \log_2 N / \log_2 M$, will be smaller if M is larger; but the work to locate each element among $M + 1$ subintervals will be larger, scaling as $\log_2 M$ for bisection, say. Each round requires looking at all N elements, if only to find those that are still alive, while the bisections are dominated by the N that occur in the first round. Minimizing $O(N \log_M N) + O(N \log_2 M)$ thus yields the result

$$M \sim 2^{\sqrt{\log_2 N}} \tag{8.5.3}$$

The square root of the logarithm is so slowly varying that secondary considerations of machine timing become important. We use $M = 64$ as a convenient constant value.

Further discussion, and code, is in a Webnote [5].

CITED REFERENCES AND FURTHER READING:

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), pp. 126ff.[1]

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley).

Chambers, J.M., James, D.A., Lambert, D., and Vander Wiel, S. 2006, "Monitoring Networked Applications with Incremental Quantiles," *Statistical Science*, vol. 21.[2]

Tieney, L. 1983, "A Space-efficient Recursive Procedure for Estimating a Quantile of an Unknown Distribution," *SIAM Journal on Scientific and Statistical Computing*, vol. 4, pp. 706-711.[3]

Liechty, J.C., Lin, D.K.J, and McDermott, J.P. 2003, "Single-Pass Low-Storage Arbitrary Quantile Estimation for Massive Datasets," *Statistics and Computing*, vol. 13, pp. 91–100.[4]

Numerical Recipes Software 2007, "Code Listing for Selip," *Numerical Recipes Webnote No. 11*, at `http://www.nr.com/webnotes?11` [5]

8.6 Determination of Equivalence Classes

A number of techniques for sorting and searching relate to data structures whose details are beyond the scope of this book, for example, trees, linked lists, etc. These structures and their manipulations are the bread and butter of computer science, as distinct from numerical analysis, and there is no shortage of books on the subject.

In working with experimental data, we have found that one particular such manipulation, namely the determination of equivalence classes, arises sufficiently often to justify inclusion here.

The problem is this: There are N "elements" (or "data points" or whatever), numbered $0, \ldots, N-1$. You are given pairwise information about whether the elements are in the same *equivalence class* of "sameness," by whatever criterion happens to be of interest. For example, you may have a list of facts like: "Element 3 and element 7 are in the same class; element 19 and element 4 are in the same class; element 7 and element 12 are in the same class, ..." Alternatively, you may have a procedure, given the numbers of two elements j and k, for deciding whether they are in the same class or different classes. (Recall that an equivalence relation can be anything satisfying the *RST properties*: reflexive, symmetric, transitive. This is compatible with any intuitive definition of "sameness.")

The desired output is an assignment to each of the N elements of an equivalence class number, such that two elements are in the same class if and only if they are assigned the same class number.

Efficient algorithms work like this: Let $F(j)$ be the class or "family" number of element j. Start off with each element in its own family, so that $F(j) = j$. The array $F(j)$ can be interpreted as a tree structure, where $F(j)$ denotes the parent of j. If we arrange for each family to be its own tree, disjoint from all the other "family trees," then we can label each family (equivalence class) by its most senior great-great-...grandparent. The detailed topology of the tree doesn't matter at all, as long as we graft each related element onto it *somewhere*.

Therefore, we process each elemental datum "j is equivalent to k" by (i) tracking j up to its highest ancestor; (ii) tracking k up to its highest ancestor; and (iii) giving j to k as a new parent, or vice versa (it makes no difference). After processing all the relations, we go through all the elements j and reset their $F(j)$'s to their highest possible ancestors, which then label the equivalence classes.

The following routine, based on Knuth [1], assumes that there are `m` elemental pieces of information, stored in two arrays of length `m`, `lista,listb`, the interpretation being that `lista[j]` and `listb[j]`, `j=0...m-1`, are the numbers of two elements that (we are thus told) are related.

eclass.h

```
void eclass(VecInt_O &nf, VecInt_I &lista, VecInt_I &listb)
```

Given `m` equivalences between pairs of `n` individual elements in the form of the input arrays `lista[0..m-1]` and `listb[0..m-1]`, this routine returns in `nf[0..n-1]` the number of the equivalence class of each of the `n` elements, integers between `0` and `n-1` (not all such integers used).

```
{
    Int l,k,j,n=nf.size(),m=lista.size();
    for (k=0;k<n;k++) nf[k]=k;                  Initialize each element its own class.
    for (l=0;l<m;l++) {                         For each piece of input information...
        j=lista[l];
        while (nf[j] != j) j=nf[j];             Track first element up to its ancestor.
        k=listb[l];
        while (nf[k] != k) k=nf[k];             Track second element up to its ancestor.
        if (j != k) nf[j]=k;                    If they are not already related, make them
    }                                               so.
    for (j=0;j<n;j++)                           Final sweep up to highest ancestors.
        while (nf[j] != nf[nf[j]]) nf[j]=nf[nf[j]];
}
```

Alternatively, we may be able to construct a boolean function `equiv(j,k)` that returns a value `true` if elements `j` and `k` are related, or `false` if they are not. Then we want to loop over all pairs of elements to get the complete picture. D. Eardley has devised a clever way of doing this while simultaneously sweeping the tree up to high ancestors in a manner that keeps it current and obviates most of the final sweep phase:

eclass.h

```
void eclazz(VecInt_O &nf, Bool equiv(const Int, const Int))
```

Given a user-supplied boolean function `equiv` that tells whether a pair of elements, each in the range `0...n-1`, are related, return in `nf[0..n-1]` equivalence class numbers for each element.

```
{
    Int kk,jj,n=nf.size();
    nf[0]=0;
    for (jj=1;jj<n;jj++) {                      Loop over first element of all pairs.
```

```
            nf[jj]=jj;
            for (kk=0;kk<jj;kk++) {                Loop over second element of all pairs.
                nf[kk]=nf[nf[kk]];                 Sweep it up this much.
                if (equiv(jj+1,kk+1)) nf[nf[nf[kk]]]=jj;
                Good exercise for the reader to figure out why this much ancestry is necessary!
            }
        }
        for (jj=0;jj<n;jj++) nf[jj]=nf[nf[jj]];        Only this much sweeping is needed
    }                                                      finally.
```

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Fundamental Algorithms*, 3rd ed., vol. 1 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §2.3.3.[1]

Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 30.

CHAPTER 9

Root Finding and Nonlinear Sets of Equations

9.0 Introduction

We now consider that most basic of tasks, solving equations numerically. While most equations are born with both a right-hand side and a left-hand side, one traditionally moves all terms to the left, leaving

$$f(x) = 0 \tag{9.0.1}$$

whose solution or solutions are desired. When there is only one independent variable, the problem is *one-dimensional*, namely to find the root or roots of a function.

With more than one independent variable, more than one equation can be satisfied simultaneously. You likely once learned the *implicit function theorem*, which (in this context) gives us the hope of satisfying N equations in N unknowns simultaneously. Note that we have only hope, not certainty. A nonlinear set of equations may have no (real) solutions at all. Contrariwise, it may have more than one solution. The implicit function theorem tells us that "generically" the solutions will be distinct, pointlike, and separated from each other. If, however, life is so unkind as to present you with a nongeneric, i.e., degenerate, case, then you can get a continuous family of solutions. In vector notation, we want to find one or more N-dimensional solution vectors $\mathbf{x}$ such that

$$\mathbf{f}(\mathbf{x}) = \mathbf{0} \tag{9.0.2}$$

where $\mathbf{f}$ is the N-dimensional vector-valued function whose components are the individual equations to be satisfied simultaneously.

Don't be fooled by the apparent notational similarity of equations (9.0.2) and (9.0.1). Simultaneous solution of equations in N dimensions is *much* more difficult than finding roots in the one-dimensional case. The principal difference between one and many dimensions is that, in one dimension, it is possible to bracket or "trap" a root between bracketing values, and then hunt it down like a rabbit. In multidimensions, you can never be sure that the root is there at all until you have found it.

Except in linear problems, root finding invariably proceeds by iteration, and this is equally true in one or in many dimensions. Starting from some approximate trial solution, a useful algorithm will improve the solution until some predetermined convergence criterion is satisfied. For smoothly varying functions, good algorithms will always converge, *provided* that the initial guess is good enough. Indeed one can even determine in advance the rate of convergence of most algorithms.

It cannot be overemphasized, however, how crucially success depends on having a good first guess for the solution, especially for multidimensional problems. This crucial beginning usually depends on analysis rather than numerics. Carefully crafted initial estimates reward you not only with reduced computational effort, but also with understanding and increased self-esteem. Hamming's motto, "the purpose of computing is insight, not numbers," is particularly apt in the area of finding roots. You should repeat this motto aloud whenever your program converges, with sixteen-digit accuracy, to the wrong root of a problem, or whenever it fails to converge because there is actually *no* root, or because there is a root but your initial estimate was not sufficiently close to it.

"This talk of insight is all very well, but what do I actually do?" For one-dimensional root finding, it is possible to give some straightforward answers: You should try to get some idea of what your function looks like before trying to find its roots. If you need to mass-produce roots for many different functions, then you should at least know what some typical members of the ensemble look like. Next, you should always bracket a root, that is, know that the function changes sign in an identified interval, before trying to converge to the root's value.

Finally (this is advice with which some daring souls might disagree, but we give it nonetheless) never let your iteration method get outside of the best bracketing bounds obtained at any stage. We will see below that some pedagogically important algorithms, such as the *secant method* or *Newton-Raphson*, can violate this last constraint and are thus not recommended unless certain fixups are implemented.

Multiple roots, or very close roots, are a real problem, especially if the multiplicity is an even number. In that case, there may be no readily apparent sign change in the function, so the notion of bracketing a root — and maintaining the bracket — becomes difficult. We are hard-liners: We nevertheless insist on bracketing a root, even if it takes the minimum-searching techniques of Chapter 10 to determine whether a tantalizing dip in the function really does cross zero. (You can easily modify the simple golden section routine of §10.2 to return early if it detects a sign change in the function. And, if the minimum of the function is exactly zero, then you have found a *double* root.)

As usual, we want to discourage you from using routines as black boxes without understanding them. However, as a guide to beginners, here are some reasonable starting points:

- Brent's algorithm in §9.3 is the method of choice to find a bracketed root of a general one-dimensional function, when you cannot easily compute the function's derivative. Ridders' method (§9.2) is concise, and a close competitor.
- When you can compute the function's derivative, the routine `rtsafe` in §9.4, which combines the Newton-Raphson method with some bookkeeping on the bounds, is recommended. Again, you must first bracket your root. If you can easily compute *two* derivatives, then Halley's method (§9.4.2) is often worth a try.

- Roots of polynomials are a special case. Laguerre's method, in §9.5, is recommended as a starting point. Beware: Some polynomials are ill-conditioned!
- Finally, for multidimensional problems, the only elementary method is Newton-Raphson (§9.6), which works *very* well if you can supply a good first guess of the solution. Try it. Then read the more advanced material in §9.7 for some more complicated, but globally more convergent, alternatives.

The routines in this chapter require that you input the function whose roots you seek. For maximum flexibility, the routines typically will accept either a function or a functor (see §1.3.3).

9.0.1 Graphical Search for Roots

It never hurts to *look at your function*, especially if you encounter any difficulty in finding its roots blindly. If you are thus hunting roots "by eye," it is useful to have a routine that repeatedly plots a function to the screen, accepting user-supplied lower and upper limits for x, automatically scaling y, and making zero crossings visible. The following routine, or something like it, can occasionally save you a lot of grief.

scrsho.h

```
template<class T>
void scrsho(T &fx) {
```

Graph the function or functor `fx` over the prompted-for interval `x1,x2`. Query for another plot until the user signals satisfaction.

```
    const Int RES=500;                          Number of function evaluations for each plot.
    const Doub XLL=75., XUR=525., YLL=250., YUR=700.;      Corners of plot, in points.
    char *plotfilename = tmpnam(NULL);
    VecDoub xx(RES), yy(RES);
    Doub x1,x2;
    Int i;
    for (;;) {
        Doub ymax = -9.99e99, ymin = 9.99e99, del;
        cout << endl << "Enter x1 x2 (x1=x2 to stop):" << endl;
        cin >> x1 >> x2;                        Query for another plot, quit if x1=x2.
        if (x1==x2) break;
        for (i=0;i<RES;i++) {                   Evaluate the function at equal intervals. Find
            xx[i] = x1 + i*(x2-x1)/(RES-1.);        the largest and smallest values.
            yy[i] = fx(xx[i]);
            if (yy[i] > ymax) ymax=yy[i];
            if (yy[i] < ymin) ymin=yy[i];
        }
        del = 0.05*((ymax-ymin)+(ymax==ymin ? abs(ymax) : 0.));
```

Plot commands, following, are in `PSplot` syntax (§22.1). You can substitute commands for your favorite plotting package.

```
        PSpage pg(plotfilename);
        PSplot plot(pg,XLL,XUR,YLL,YUR);
        plot.setlimits(x1,x2,ymin-del,ymax+del);
        plot.frame();
        plot.autoscales();
        plot.lineplot(xx,yy);
        if (ymax*ymin < 0.) plot.lineseg(x1,0.,x2,0.);
        plot.display();
    }
    remove(plotfilename);
}
```

CITED REFERENCES AND FURTHER READING:

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 5.

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapters 2, 7, and 14.

Deuflhard, P. 2004, *Newton Methods for Nonlinear Problems* (Berlin: Springer).

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 8.

Householder, A.S. 1970, *The Numerical Treatment of a Single Nonlinear Equation* (New York: McGraw-Hill).

9.1 Bracketing and Bisection

We will say that a root is *bracketed* in the interval (a, b) if $f(a)$ and $f(b)$ have opposite signs. If the function is continuous, then at least one root must lie in that interval (the *intermediate value theorem*). If the function is discontinuous, but bounded, then instead of a root there might be a step discontinuity that crosses zero (see Figure 9.1.1). For numerical purposes, that might as well be a root, since the behavior is indistinguishable from the case of a continuous function whose zero crossing occurs in between two "adjacent" floating-point numbers in a machine's finite-precision representation. Only for functions with singularities is there the possibility that a bracketed root is not really there, as for example

$$f(x) = \frac{1}{x - c} \tag{9.1.1}$$

Some root-finding algorithms (e.g., bisection in this section) will readily converge to c in (9.1.1). Luckily there is not much possibility of your mistaking c, or any number x close to it, for a root, since mere evaluation of $|f(x)|$ will give a very large, rather than a very small, result.

If you are given a function in a black box, there is no sure way of bracketing its roots, or even of determining that it has roots. If you like pathological examples, think about the problem of locating the two real roots of equation (3.0.1), which dips below zero only in the ridiculously small interval of about $x = \pi \pm 10^{-667}$.

In the next chapter we will deal with the related problem of bracketing a function's minimum. There it is possible to give a procedure that always succeeds; in essence, "Go downhill, taking steps of increasing size, until your function starts back uphill." There is no analogous procedure for roots. The procedure "go downhill until your function changes sign," can be foiled by a function that has a simple extremum. Nevertheless, if you are prepared to deal with a "failure" outcome, this procedure is often a good first start; success is usual if your function has opposite signs in the limit $x \to \pm\infty$.

roots.h

```
template <class T>
Bool zbrac(T &func, Doub &x1, Doub &x2)
```

Given a function or functor `func` and an initial guessed range `x1` to `x2`, the routine expands the range geometrically until a root is bracketed by the returned values `x1` and `x2` (in which case `zbrac` returns `true`) or until the range becomes unacceptably large (in which case `zbrac` returns `false`).

```
{
    const Int NTRY=50;
    const Doub FACTOR=1.6;
```

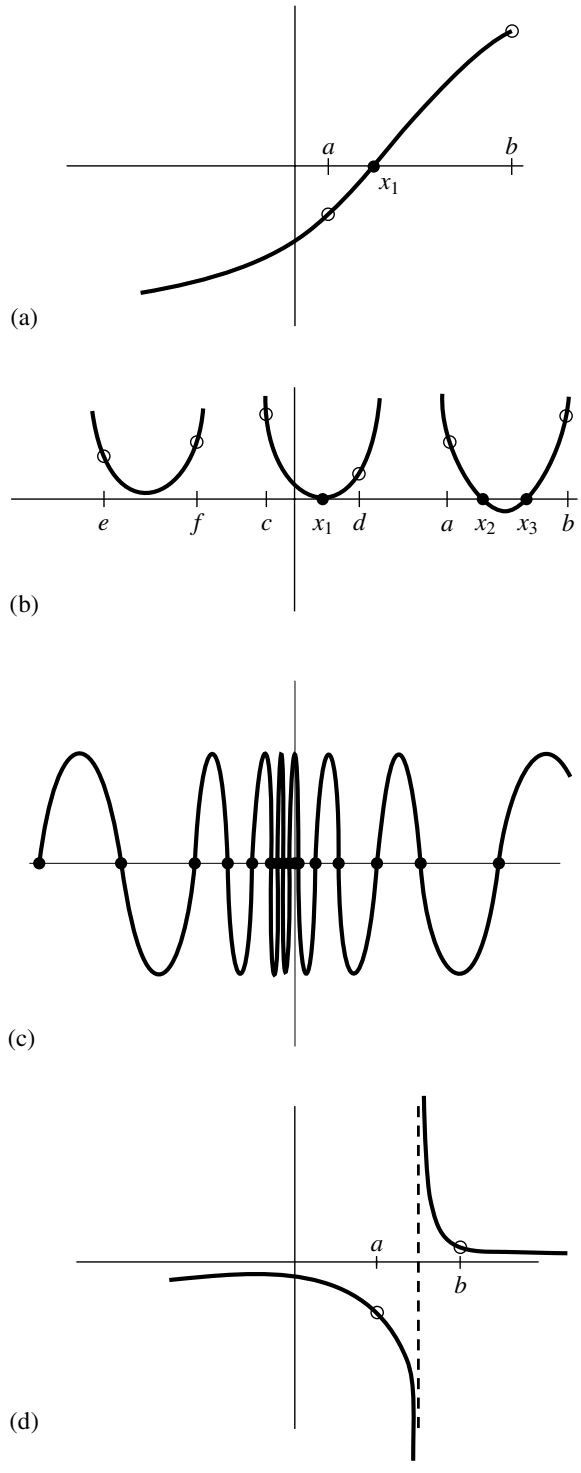

Figure 9.1.1. Some situations encountered while root finding: (a) an isolated root x_1 bracketed by two points a and b at which the function has opposite signs; (b) there is not necessarily a sign change in the function near a double root (in fact, there is not necessarily a root!); (c) a pathological function with many roots; in (d) the function has opposite signs at points a and b, but the points bracket a singularity, not a root.

```
    if (x1 == x2) throw("Bad initial range in zbrac");
    Doub f1=func(x1);
    Doub f2=func(x2);
    for (Int j=0;j<NTRY;j++) {
        if (f1*f2 < 0.0) return true;
        if (abs(f1) < abs(f2))
            f1=func(x1 += FACTOR*(x1-x2));
        else
            f2=func(x2 += FACTOR*(x2-x1));
    }
    return false;
}
```

Alternatively, you might want to "look inward" on an initial interval, rather than "look outward" from it, asking if there are any roots of the function $f(x)$ in the interval from x_1 to x_2 when a search is carried out by subdivision into n equal intervals. The following function calculates brackets for distinct intervals that each contain one or more roots.

roots.h

```
template <class T>
void zbrak(T &fx, const Doub x1, const Doub x2, const Int n, VecDoub_O &xb1,
    VecDoub_O &xb2, Int &nroot)
```

Given a function or functor fx defined on the interval [x1,x2], subdivide the interval into n equally spaced segments, and search for zero crossings of the function. nroot will be set to the number of bracketing pairs found. If it is positive, the arrays xb1[0..nroot-1] and xb2[0..nroot-1] will be filled sequentially with any bracketing pairs that are found. On input, these vectors may have any size, including zero; they will be resized to $\geq$ nroot.

```
{
    Int nb=20;
    xb1.resize(nb);
    xb2.resize(nb);
    nroot=0;
    Doub dx=(x2-x1)/n;                  Determine the spacing appropriate to the mesh.
    Doub x=x1;
    Doub fp=fx(x1);
    for (Int i=0;i<n;i++) {             Loop over all intervals
        Doub fc=fx(x += dx);
        if (fc*fp <= 0.0) {             If a sign change occurs, then record values for the
            xb1[nroot]=x-dx;                bounds.
            xb2[nroot++]=x;
            if(nroot == nb) {
                VecDoub tempvec1(xb1),tempvec2(xb2);
                xb1.resize(2*nb);
                xb2.resize(2*nb);
                for (Int j=0; j<nb; j++) {
                    xb1[j]=tempvec1[j];
                    xb2[j]=tempvec2[j];
                }
                nb *= 2;
            }
        }
        fp=fc;
    }
}
```

9.1.1 Bisection Method

Once we know that an interval contains a root, several classical procedures are available to refine it. These proceed with varying degrees of speed and sure-

ness toward the answer. Unfortunately, the methods that are guaranteed to converge plod along most slowly, while those that rush to the solution in the best cases can also dash rapidly to infinity without warning if measures are not taken to avoid such behavior.

The *bisection method* is one that cannot fail. It is thus not to be sneered at as a method for otherwise badly behaved problems. The idea is simple. Over some interval the function is known to pass through zero because it changes sign. Evaluate the function at the interval's midpoint and examine its sign. Use the midpoint to replace whichever limit has the same sign. After each iteration the bounds containing the root decrease by a factor of two. If after n iterations the root is known to be within an interval of size ϵ_n, then after the next iteration it will be bracketed within an interval of size

$$\epsilon_{n+1} = \epsilon_n/2 \tag{9.1.2}$$

neither more nor less. Thus, we know in advance the number of iterations required to achieve a given tolerance in the solution,

$$n = \log_2 \frac{\epsilon_0}{\epsilon} \tag{9.1.3}$$

where ϵ_0 is the size of the initially bracketing interval and ϵ is the desired ending tolerance.

Bisection *must* succeed. If the interval happens to contain more than one root, bisection will find one of them. If the interval contains no roots and merely straddles a singularity, it will converge on the singularity.

When a method converges as a factor (less than 1) times the previous uncertainty to the first power (as is the case for bisection), it is said to converge *linearly*. Methods that converge as a higher power,

$$\epsilon_{n+1} = \text{constant} \times (\epsilon_n)^m \qquad m > 1 \tag{9.1.4}$$

are said to converge superlinearly. In other contexts, "linear" convergence would be termed "exponential" or "geometrical." That is not too bad at all: Linear convergence means that successive significant figures are won linearly with computational effort.

It remains to discuss practical criteria for convergence. It is crucial to keep in mind that only a finite set of floating point values have exact computer representations. While your function might analytically pass through zero, it is probable that its computed value is never zero, for any floating-point argument. One must decide what accuracy on the root is attainable: Convergence to within 10^{-10} in absolute value is reasonable when the root lies near 1 but certainly unachievable if the root lies near 10^{26}. One might thus think to specify convergence by a relative (fractional) criterion, but this becomes unworkable for roots near zero. To be most general, the routines below will require you to specify an absolute tolerance, such that iterations continue until the interval becomes smaller than this tolerance in absolute units. Often you may wish to take the tolerance to be $\epsilon(|x_1| + |x_2|)/2$, where ϵ is the machine precision and x_1 and x_2 are the initial brackets. When the root lies near zero you ought to consider carefully what reasonable tolerance means for your function. The following routine quits after 50 bisections in any event, with $2^{-50} \approx 10^{-15}$.